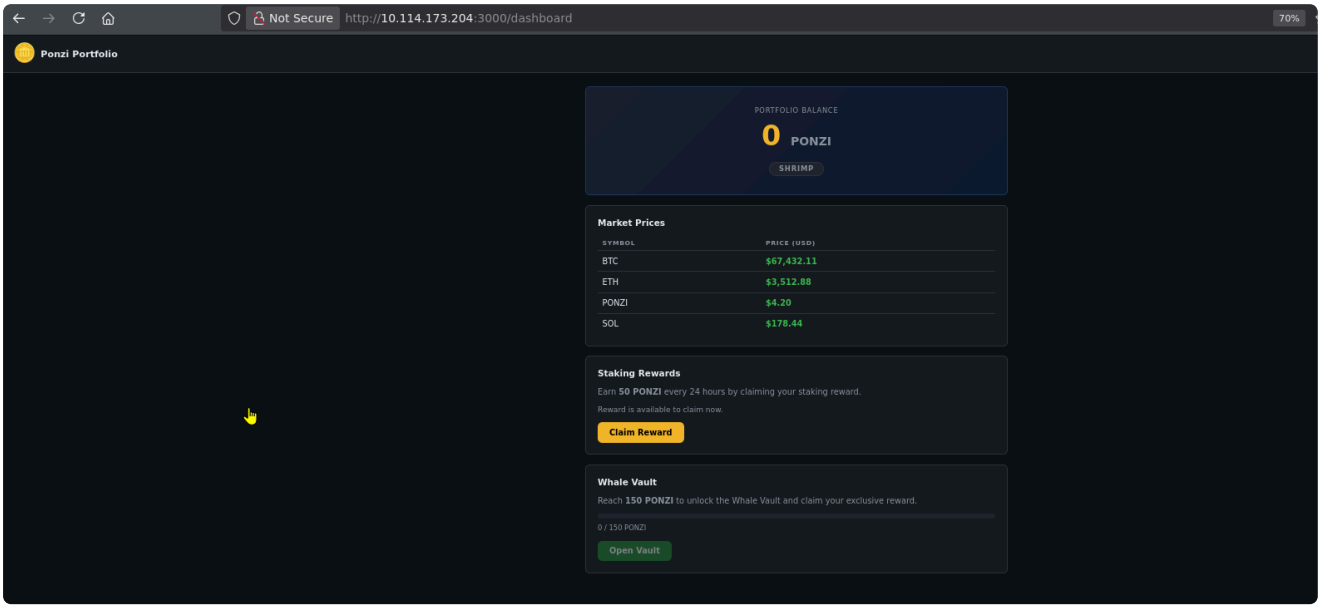
#WebExploitation #BusinessLogic #BurpSuite #APIAbuse

Objective

The challenge required me to abuse the application's staking reward logic and obtain enough **PONZI** to unlock the **Whale Vault** and retrieve the flag.
The portfolio initially contained **0 PONZI**. The application offered a staking reward of **50 PONZI every 24 hours**, while the Whale Vault required **150 PONZI** to unlock.

1. Inspecting the staking functionality

I opened the portfolio dashboard and reviewed the available functionality.



Burp captured a request to the application's claim endpoint:

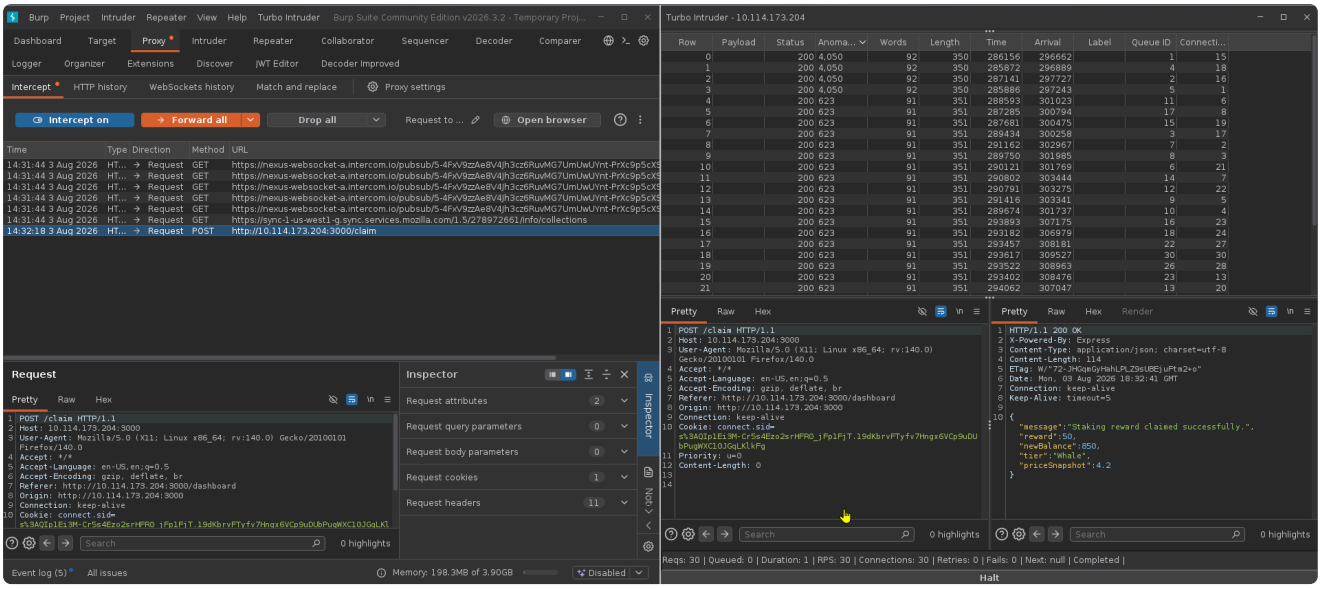
```
POST /claim HTTP/1.1
Host: 10.114.173.204:3000
```

The request did not require any additional body parameters. Authentication was handled by the existing session cookie.

The server response confirmed that a successful request awarded **50 PONZI**:

```
{
  "message": "Staking reward claimed successfully.",
  "reward": 50,
  "newBalance": 850,
  "priceSnapshot": 4.2
}
```

This showed that the `/claim` endpoint directly modified the user's portfolio balance.



3. Testing the claim endpoint for a race condition

The application was supposed to allow only one staking reward every 24 hours.

I sent the captured `/claim` request to **Turbo Intruder** and submitted multiple requests against the endpoint concurrently.

The attack queued **30 requests** with multiple simultaneous connections.

Instead of accepting only one request and rejecting the remaining requests because of the cooldown, the application returned successful responses to the concurrent claims.

Each successful request awarded another:

```
50 PONZI
```

This indicated a **race condition in the application's business logic**.

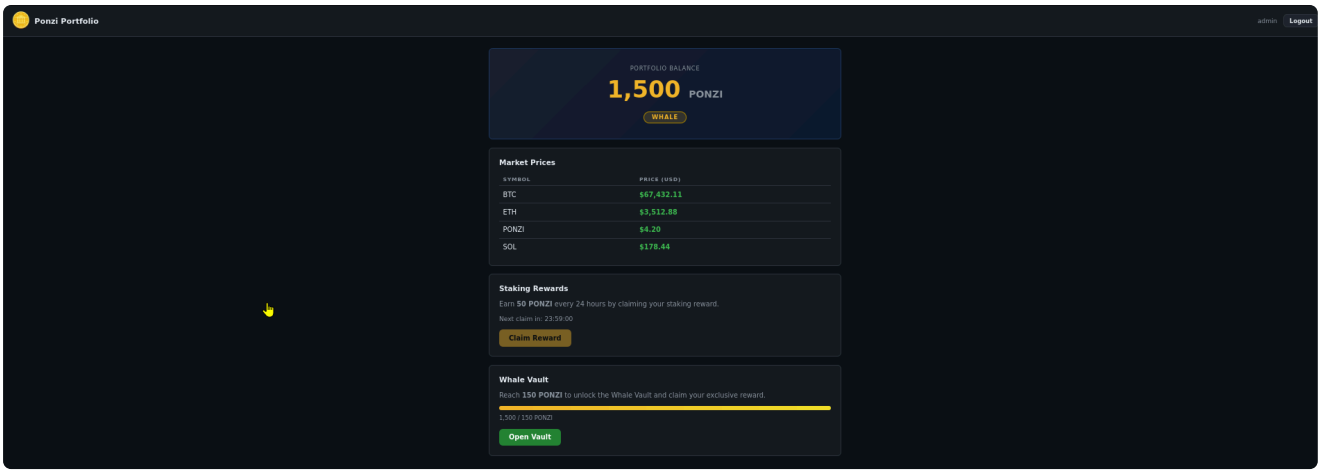
The application was checking whether the reward could be claimed and updating the balance without properly preventing multiple requests from performing the same operation at the same time.

As a result, several requests passed the reward validation before the application recorded that the reward had already been claimed.

4. Increasing the portfolio balance

After the concurrent requests completed, I refreshed the portfolio dashboard.

The balance had increased, because each successful request awarded **50 PONZI**, the concurrent requests allowed the reward to be claimed repeatedly without waiting for the normal 24-hour cooldown.



I opened the Whale Vault after reaching the required balance.

The application revealed the challenge flag:

```
THM{t0w3l_0n_th3_sunb3d_d0ubl3_sp3nt}
```

The vulnerability was therefore caused by a **race condition / double-spending flaw** in the reward-claiming logic.